

Q1. What is a Virtual Machine?

Imagine you write a letter in English, but the person who receives it understands only Hindi. So you need a translator in between.

Similarly:

Your program is written in a programming language.

The computer understands only machine language (0s and 1s).

So we need something in between that converts your program into machine language.

That “something” is called a Virtual Machine.

A Virtual Machine (VM) is a software program that behaves like a real computer. It runs programs as if it were an actual machine.

Simple Example

Suppose:

You write a Java program on Windows.

Your friend wants to run it on Mac or Linux.

Normally, programs made for Windows do not run on Mac.

But Java programs run everywhere.

Why?

Because of a Virtual Machine called the JVM.

What is JVM?

JVM stands for Java Virtual Machine.

JVM is a virtual computer that runs Java programs.

When you write a Java program:

1. You write code in .java file.
2. The Java compiler converts it into bytecode (.class file).
3. This bytecode is not machine language.
4. JVM converts this bytecode into machine language.
5. Then the computer executes it.

How Java Program Runs

Java Program (.java)



Compiler (javac)



Bytecode (.class)



JVM



Machine Code



Output

Why JVM is Important?

Platform Independent

Java follows:

“Write Once, Run Anywhere (WORA)”

Because JVM is available for Windows, Mac, Linux.

Memory Management : JVM automatically manages memory (Garbage Collection).

Security: JVM checks the code before running.

Portability: Same .class file runs on any system with JVM.

In Simple Words

JVM is like a translator + manager that:

1. Converts bytecode to machine code
2. Manages memory
3. Provides security
4. Makes Java platform independent

Q2. What is JRE?

JRE stands for **Java Runtime Environment**.

JRE is the complete environment required to **run** a Java program.

Think of it like this:

If JVM is the **engine**,
then JRE is the **car** (engine + fuel + wheels + everything needed to run).

Very Simple Example

Suppose you download a Java game.

To play the game:

- You do NOT need to write code.
- You do NOT need a compiler.
- You just need something that can **run** the program.

That “something” is **JRE**.

What Does JRE Contain?

JRE includes:

1. **JVM** – to run the bytecode
2. **Java Class Libraries** – ready-made classes like:
 - String
 - System
 - Math
 - Scanner

These libraries help the program run properly.

Simple Structure

JRE = JVM + Java Libraries + Supporting Files

Easy Difference Between JVM and JRE

JVM	JRE
Runs bytecode	Provides complete environment to run Java programs
Part of JRE	Contains JVM
Only execution engine	Execution + Libraries

In Very Simple Words

JVM is the **heart**

JRE is the **body (heart + other parts)**

Without JVM → program cannot run

Without JRE → JVM cannot function properly

Q3. What is JDK?

JDK stands for **Java Development Kit**.

JDK is used to **develop (write + compile + run)** Java programs.

If JRE is for running programs,
then JDK is for creating programs.

Very Simple Example

Suppose:

- You only want to **watch a movie** → You need a media player (JRE).
- But if you want to **create a movie** → You need editing software (JDK).

Similarly:

- To only run Java programs → Install **JRE**
- To write and create Java programs → Install **JDK**

What Does JDK Contain?

JDK includes:

1. **JRE**
2. **JVM**
3. **Compiler (javac)**
4. Development tools (debugger, documentation tool, etc.)

Simple Structure

JDK = JRE + Development Tools

JRE = JVM + Libraries

So finally:

JDK = JVM + Libraries + Compiler + Tools

Simple Comparison

Component Used For

JVM	Runs bytecode
JRE	Runs Java programs
JDK	Develops + Compiles + Runs Java programs

One-Line Understanding

JVM = Engine

RE = Engine + Running Environment

JDK = Everything needed to create and run Java programs

Now your students clearly understand the full hierarchy:

JDK

└─ JRE

└─ JVM

Q4 . What is a Class?

Think very simple.

A **class** is a **blueprint (design)**.

It tells:

- What data something has
- What actions it can perform

But class itself is **not real**.

It is only a design.

Simple Example 1 – Student Class

Suppose we want to describe a student.

Every student has:

- name
- roll number
- marks

Every student can:

- study
- give exam

So we create a **Student class** that defines these things.

But remember:

Class is only a **plan**, not an actual student.

Simple Example 2 – Car Class

A car has:

- color
- speed
- model

A car can:

- start
- stop
- accelerate

So we create a **Car class**.

Again — this is only a design.

Q5. What is an Object?

An **object** is a real thing created from a class or it's a real world entity.

If class is a blueprint,
object is the real building.

If class is a design of car,
object is the real car.

Example

Student class → defines structure

Ajay (student1) → object

Car class → defines structure

MyCar → object

You can create many objects from one class.

Difference in Very Simple Words

Class	Object
Blueprint	Real thing
Logical	Physical
Defines data and actions	Uses data and actions

Small Example: Box Class

Now let us create a simple class called **Box**.

A box has:

- length
- width
- height

And it can:

- calculate volume

Java Program

```
class Box {  
    // Data (variables) - Properties  
    double length;  
    double width;  
    double height;  
  
    // Method (function)  
    double calculateVolume() {  
        return length * width * height;  
    }  
    public static void main(String[] args) {  
        // Creating object  
        Box box1 = new Box();  
        // Assigning values  
        box1.length = 5;  
        box1.width = 4;  
        box1.height = 3;  
        // Calling method  
        double volume = box1.calculateVolume();  
        System.out.println("Volume of Box = " + volume);  
    }  
}
```

```
}
```

Explanation in Very Simple Words

class Box

We created a blueprint named Box.

Instance variables

double length;

double width;

double height;

These are data members.

They store information about the box.

Instance method Method

double calculateVolume()

This is an action. It calculates volume.

Object Creation

Box box1 = new Box();

Here we created a real box object from the class.

Assign Values

We gave dimensions to the box.

Call Method

We called the method to calculate volume.

Final Understanding

Class = Design

Object = Real thing created from design

Variables = Data

Methods = Actions

This is the heart of Object-Oriented Programming.

[Q6. Explain the process of compiling and running a Java program. Also explain how JVM executes the program.](#)

1. Writing the Java Program

First, the programmer writes the Java program using a **text editor or an IDE** such as VSCode.

The program is saved with the **.java** extension.

Example: MyProgram.java

Example program:

```
class MyProgram {  
    public static void main(String args[]) {  
        System.out.println("Hello Java");  
    }  
}
```

2. Compiling the Java Program

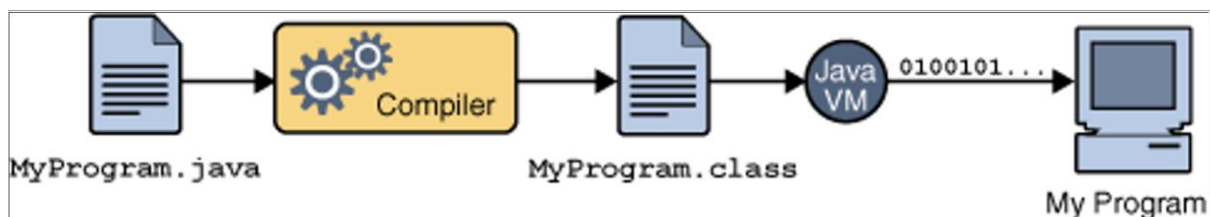
Compilation means **converting the Java source code into bytecode**.

Steps:

1. Open **Command Prompt**.
2. Go to the folder where the Java file is saved.
3. Use the **javac** compiler command.

Example: `javac MyProgram.java`

After compilation, a new file is created: `MyProgram.class`



3. Running the Java Program

After successful compilation, the program can be executed using the **java** command.

Example: `java MyProgram`

Output: Hello Java

The JVM reads the **.class file** and runs the program.

Q7 . Explain the important features of Java.

Some important features of Java are given below:

1. **Simple**
Java is easy to learn and understand. Its syntax is similar to C and C++, but it removes complicated features such as pointers. This makes programming easier for beginners.
2. **Object-Oriented**
Java follows the object-oriented approach. In Java, programs are written using **classes and objects**, which helps in organizing programs and reusing code.

3. **Platform Independent**

Java programs can run on different operating systems such as Windows, Linux, and Mac. Java code is first converted into **bytecode**, which can run on any system that has the **Java Virtual Machine (JVM)**. Therefore, Java follows the principle “**Write Once, Run Anywhere.**”

4. **Secure**

Java provides a secure programming environment. It does not allow direct access to memory through pointers, which reduces the chances of errors and security problems.

5. **Robust (Reliable)**

Java is a reliable language. It has features like **automatic memory management** and **error handling**, which help reduce program crashes.

6. **Portable**

Java programs can easily move from one system to another without major changes because Java uses standard data types and bytecode.